Übungsblatt 7

zur Vorlesung Efficient Computing/Parallel Computing 2 im SS 2026

Datum der Ausgabe: 19.5.2026 Datum der Abgabe: 15.6.2026

Die mit ★ gekennzeichneten Aufgaben sind für die Prüfungszulassung relevant und unter Angabe von Namen und Matrikelnummer in elektronischer Form zum Abgabetermin in Moodle einzureichen.

★ **Aufgabe 7.1** (*OpenMP-parallelized n-body problem using the Barnes-Hut algorithm*).

Consider the gravitational forces between n particles of varying mass in two space dimensions. To every particle i, associate a mass m_i as well as coordinate and velocity vector denoted by

$$\mathbf{c}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix} \quad \text{and} \quad \mathbf{v}_i = \begin{bmatrix} v_i^x \\ v_i^y \end{bmatrix},$$

respectively. The gravitational force on mass m_i by a single mass m_j is then given by

$$\frac{Gm_i m_j(\mathbf{c}_j - \mathbf{c}_i)}{||\mathbf{c}_j - \mathbf{c}_i||^3},$$

where G is the gravitational constant. The product of mass and acceleration of particle i is obtained by summing the gravitational forces over all particles leading to

$$m_i \frac{\mathrm{d}\mathbf{v}_i}{\mathrm{d}t} = \sum_{\substack{j=1 \\ i \neq j}}^{n} \frac{Gm_i m_j(\mathbf{c}_j - \mathbf{c}_i)}{||\mathbf{c}_j - \mathbf{c}_i||^3}. \tag{1}$$

Furthermore, the position and velocity of particle i are related by

$$\frac{\mathrm{d}\mathbf{c}_i}{\mathrm{d}t} = \mathbf{v}_i. \tag{2}$$

The explicit Euler method is among the simplest approaches to integrate equations of the form (1) or (2). Starting at some time t_0, this method discretizes time on an equidistant grid

$$t^{(k)} = t_0 + k\Delta t, \quad k = 0, 1, 2, \ldots,$$

where Δt is the size of the time step and the superscript denotes the index of the time step. Using the explicit Euler method, the velocity at the next time step is given by

$$\mathbf{v}_i^{(k+1)} = \mathbf{v}_i^{(k)} + \Delta t \sum_{\substack{j=1 \\ i \neq j}}^{n} \frac{Gm_j(\mathbf{c}_j - \mathbf{c}_i)}{||\mathbf{c}_j - \mathbf{c}_i||^3} \tag{3}$$

and the position at the next time step is given by

$$\mathbf{c}_i^{(k+1)} = \mathbf{c}_i^{(k)} + \Delta t \mathbf{v}_i^{(k)}. \tag{4}$$

In contrast to a so-called *direct method*, which consists of a straightforward sequential algorithm that considers all pairs of particles in each time step, we will now consider a more efficient implementation based on the Barnes–Hut algorithm (BHA). As discussed in the lecture, the BHA does not compute the force contributions by summing over (3) for every pair; instead it makes extensive use of a quad-tree data structure to combine particles that are close to each other but far from the current particle into a single cluster.

A serial C++ implementation of the n-body problem using the direct method is provided in `nbody.cpp` 📌 .

Barnes-Hut algorithm. Use the provided serial direct-method code as a basis to implement the BHA for 2D in C/C++. The program shall build a quad-tree from the current particle positions at each time step, traverse it to approximate the force on every particle, and advance positions and velocities via (3)–(4). Verify the correctness by comparing trajectories to the direct-method reference for a small particle count.

OpenMP parallelization. Parallelize the serial code and its modified version implementing the Barnes-Hut Algorithm with OpenMP (if possible). Identify all parallel regions and apply appropriate pragmas. Correctly classify shared and private variables. Again, verify the correctness by comparing the output.

Performance measurement. Time the time-stepping loops (excluding initialization and file I/O) of the different code versions for varying number of particles. For the parallel versions use different thread counts $p \in \{1, 2, 4, \ldots\}$.

a) Report the runtimes $T(p)$ w.r.t. thread counts p in a table.

b) Compute the speedup $S(p) = T(1)/T(p)$ and the efficiency $E(p) = S(p)/p$.

c) Plot $S(p)$ against ideal linear speedup and discuss deviations.

d) Fit Amdahl's law to estimate the serial fraction and comment on the result.

★ Aufgabe 7.2 (*OpenMP-parallelized 3D Ray Tracing*).

Ray tracing is a rendering technique that simulates the propagation of light rays through a three-dimensional scene. For each pixel (i, j) of the output image, a ray is cast from a camera position $\mathbf{C} \in \mathbb{R}^3$ in a direction $\mathbf{D} \in \mathbb{R}^3$ (with $\|\mathbf{D}\| = 1$) into the scene. The scene consists of a set of triangles, each defined by three vertices $\mathbf{T}_1, \mathbf{T}_2, \mathbf{T}_3 \in \mathbb{R}^3$ and a material color $\mathbf{k} \in [0, 1]^3$.

Given a camera position $\mathbf{C}$, a look-at point $\mathbf{P}$, and an up-vector $\mathbf{U}$, the orthonormal base vectors for the camera is constructed as

$$\mathbf{f} = \frac{\mathbf{P} - \mathbf{C}}{\|\mathbf{P} - \mathbf{C}\|}, \qquad \mathbf{r} = \frac{\mathbf{f} \times \mathbf{U}}{\|\mathbf{f} \times \mathbf{U}\|}, \qquad \mathbf{u} = \mathbf{r} \times \mathbf{f}. \tag{5}$$

For pixel (i, j) in an image of width W and height H, with field of view φ and aspect ratio $a = W/H$, the normalized screen coordinates are

$$x = \Big(2\,\frac{i + 0.5}{W} - 1\Big)\,\tan\Big(\frac{\varphi}{2}\Big)\,a, \qquad y = -\Big(2\,\frac{j + 0.5}{H} - 1\Big)\,\tan\Big(\frac{\varphi}{2}\Big).$$

The unnormalized ray direction is then

$$\mathbf{D} = \frac{x\,\mathbf{r} + y\,\mathbf{u} + \mathbf{f}}{\|x\,\mathbf{r} + y\,\mathbf{u} + \mathbf{f}\|}. \tag{6}$$

The intersection of a ray $\mathbf{C} + \alpha\,\mathbf{D}$ with a triangle $(\mathbf{T}_1, \mathbf{T}_2, \mathbf{T}_3)$ is computed using the *Möller–Trumbore algorithm*. Define the edge vectors $\mathbf{E}_1 = \mathbf{T}_2 - \mathbf{T}_1$ and $\mathbf{E}_2 = \mathbf{T}_3 - \mathbf{T}_1$. Let

$$\mathbf{U} = \mathbf{D} \times \mathbf{E}_2, \qquad \beta = \mathbf{E}_1 \cdot \mathbf{U}.$$

If $|\beta| < \varepsilon$ (with a small tolerance $\varepsilon > 0$), the ray is parallel to the triangle and no intersection exists. Otherwise, set $\mathbf{V} = \mathbf{C} - \mathbf{T}_1$ and compute the barycentric coordinates

$$\lambda_2 = \frac{\mathbf{V} \cdot \mathbf{U}}{\beta}, \qquad \lambda_3 = \frac{\mathbf{D} \cdot (\mathbf{V} \times \mathbf{E}_1)}{\beta}. \tag{7}$$

A valid intersection requires $\lambda_2 \geq 0$, $\lambda_3 \geq 0$, and $\lambda_2 + \lambda_3 \leq 1$. The ray parameter is then

$$\alpha = \frac{\mathbf{E}_2 \cdot (\mathbf{V} \times \mathbf{E}_1)}{\beta}, \tag{8}$$

and the intersection is valid only if $\alpha > \varepsilon$. The outward normal is $\mathbf{n} = (\mathbf{E}_1 \times \mathbf{E}_2)/\|\mathbf{E}_1 \times \mathbf{E}_2\|$.

For each ray, the scene is traversed to find the closest intersection (smallest $\alpha > 0$). If no intersection is found, the pixel is assigned the background color (white). Otherwise, let $\mathbf{H} = \mathbf{C} + \alpha\,\mathbf{D}$ be the hit point and $\mathbf{L}$ the light source position. The pixel color is computed using Lambert's diffuse shading model:

$$\mathbf{c}_{ij} = \mathbf{k} \cdot \max\Big(0,\ -\mathbf{n} \cdot \frac{\mathbf{H} - \mathbf{L}}{\|\mathbf{H} - \mathbf{L}\|}\Big). \tag{9}$$

The components of $\mathbf{c}_{ij} \in [0,1]^3$ are finally scaled to $\{0, \ldots, 255\}$ and written to a PPM file.

The scene geometry is provided as a ASCII STL file, a plain-text format that stores a triangulated surface. Each triangle is specified by its outward normal (which may be ignored in favour of the computed normal) followed by three vertex coordinate triples.

Serial implementation. Implement a serial version of the ray tracer in C based on the reference `SimpleRenderWithSTL.py` 📌 . The program shall read an ASCII STL file, set up the camera frame (5) with the parameters in Table 1, cast one ray per pixel (6), find the closest intersection (7)–(8), shade it (9), and write the result to a PPM file. The C output must visually match the Python reference for the provided STL `test.stl` 📌 .

OpenMP parallelization. Parallelize the serial C code with OpenMP. Identify all parallelizable regions, apply the appropriate pragmas, and correctly classify shared and private variables. Verify correctness of the parallel version by checking its visual result.

Performance measurement. Time the rendering part (excluding file I/O) for different thread counts $p \in \{1, 2, 4, \ldots\}$ and varying resolutions (e.g. 512×512 and 1024×1024).

a) Report the runtimes $T(p)$ w.r.t. thread counts p in a table.

b) Compute the speedup $S(p) = T(1)/T(p)$ and the efficiency $E(p) = S(p)/p$.

c) Plot $S(p)$ against ideal linear speedup and discuss deviations.

d) Fit Amdahl's law to estimate the serial fraction and comment on the result.

Tabelle 1: Default rendering parameters.

Parameter	Value
Camera position $\mathbf{C}$	$(20, -20, 10)^\top$
Look-at point $\mathbf{P}$	$(0, 0, 3)^\top$
Up-vector $\mathbf{u}$	$(0, 0, 1)^\top$
Light source $\mathbf{L}$	$(20, -20, 5)^\top$
Field of view φ	$\pi/3$
Background color	$(1, 1, 1)$